Wrocław University of Science and Technology
Faculty of Management
Physics of complex systems

Report on various network analyses

Hubert Nowicki

Wrocław 2026

Contents

1 Random Geometric Graph **2**
1.1 Definition 2
1.2 Graph construction 2
1.3 Visualization 4

2 Watts-Strogazt model **8**
2.1 Definition 8
2.2 Graph construction 8
2.3 Analysis 9

3 Graph analysis with NetworkX **10**
3.1 Description 10
3.2 Analysis 10
3.3 Visualization 11

1 Random Geometric Graph

1.1 Definition

A random geometric graph (RGG) is a spatial network model in which nodes are randomly distributed within a geometric region, typically in a two dimensional plane, and edges are established between pairs of nodes that lie within a predefined distance threshold. The structure of the graph therefore depends on both the spatial distribution of the nodes and the chosen connection radius. Random geometric graphs are widely used to represent systems in which connectivity is influenced by physical proximity, such as wireless communication networks, sensor networks, transportation systems, and other spatially embedded complex networks.

1.2 Graph construction

First task in the list states the following:

> Generate N points uniformly at random in the unit square $[0, 1] \times [0, 1]$. Treat each point as a node of the graph. Connect two nodes i and j with an edge if the Euclidean distance between them satisfies:
>
> $$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \leq r$$

That's why in order to create said graph, a double nested loop was implemented.

```
graph = {}
for i in range(N):
    graph.update({i: []})
print(graph)
for i in graph:
    for j in graph:
        if i != j:
            # coordinates for i node
            xi = coordinates[i][0] # value of column 0 in row i
            yi = coordinates[i][1] # value of column 1 in row i
                                   # together they make coordinates
                                      of a node

            # coordinates for j node
            xj = coordinates[j][0]
            yj = coordinates[j][1]

            d = math.sqrt((xi-xj)**2 + (yi-yj)**2)
            if d <= r:
                graph[i].append(j)
```

This generated the following output, successfully creating connections between each of the nodes:

```
{0: [11, 13, 16, 31, 36, 40, 46, 52, 55, 57, 58, 65, 70],
 1: [14, 26, 41, 42, 50, 56, 74, 85, 87, 99],
 2: [9, 10, 17, 39, 45, 62, 68, 96],
 3: [12, 30, 32, 55, 61, 66, 70, 78, 94],
 4: [6, 29, 37, 38, 69, 83],
 5: [6, 18, 21, 28, 51, 59, 60, 72, 79],
 6: [4, 5, 15, 18, 21, 29, 37, 38, 69, 72, 79, 83, 93],
 7: [8, 25, 34, 35, 53, 54, 71, 84, 88],
 8: [7, 14, 25, 26, 35, 74, 88],
 9: [2, 17, 39, 45, 62, 96],
 10: [2, 11, 16, 17, 31, 40, 42, 46, 58, 68, 99],
```

Next, the degree of each node and the average degree of the node ($\mu = 9.6$) were computed along with creating a histogram of the degree distribution.

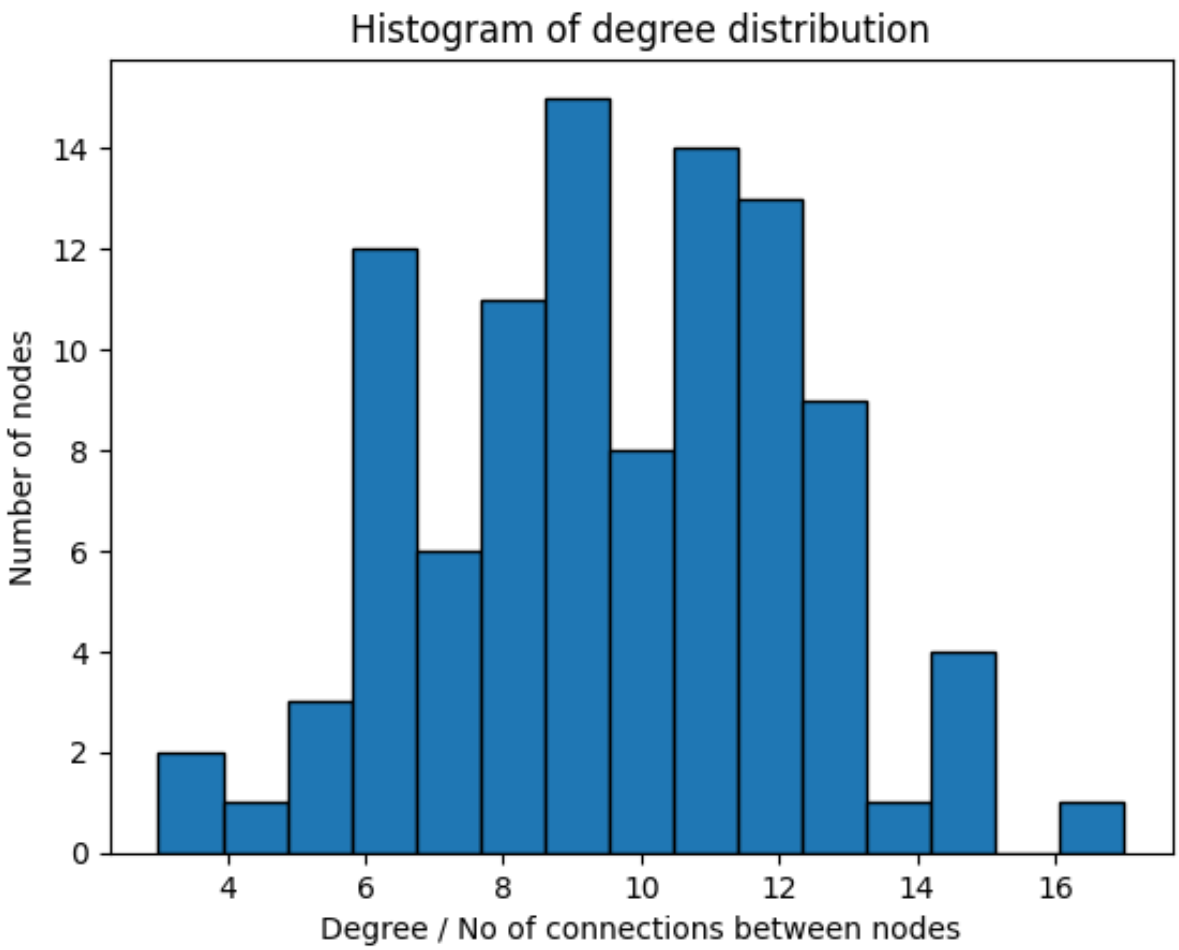

Figure 1: Histogram of the degree distribution

The obtained histogram displays a distinct, bell shaped curve that is highly characteristic of random graphs. The data indicates that the network's connections are distributed relatively evenly around a central mean, with the vast majority of nodes possessing between 6 and 12 connections and peaking prominently at a degree of roughly 9.

1.3 Visualization

To visualize properly networks, a series of graphs were created for $N = 100$ and $N = 1000$. Moreover, for each value of N two different values of r were chosen:

- one for which the graph is not connected
- one for which the graph is connected.

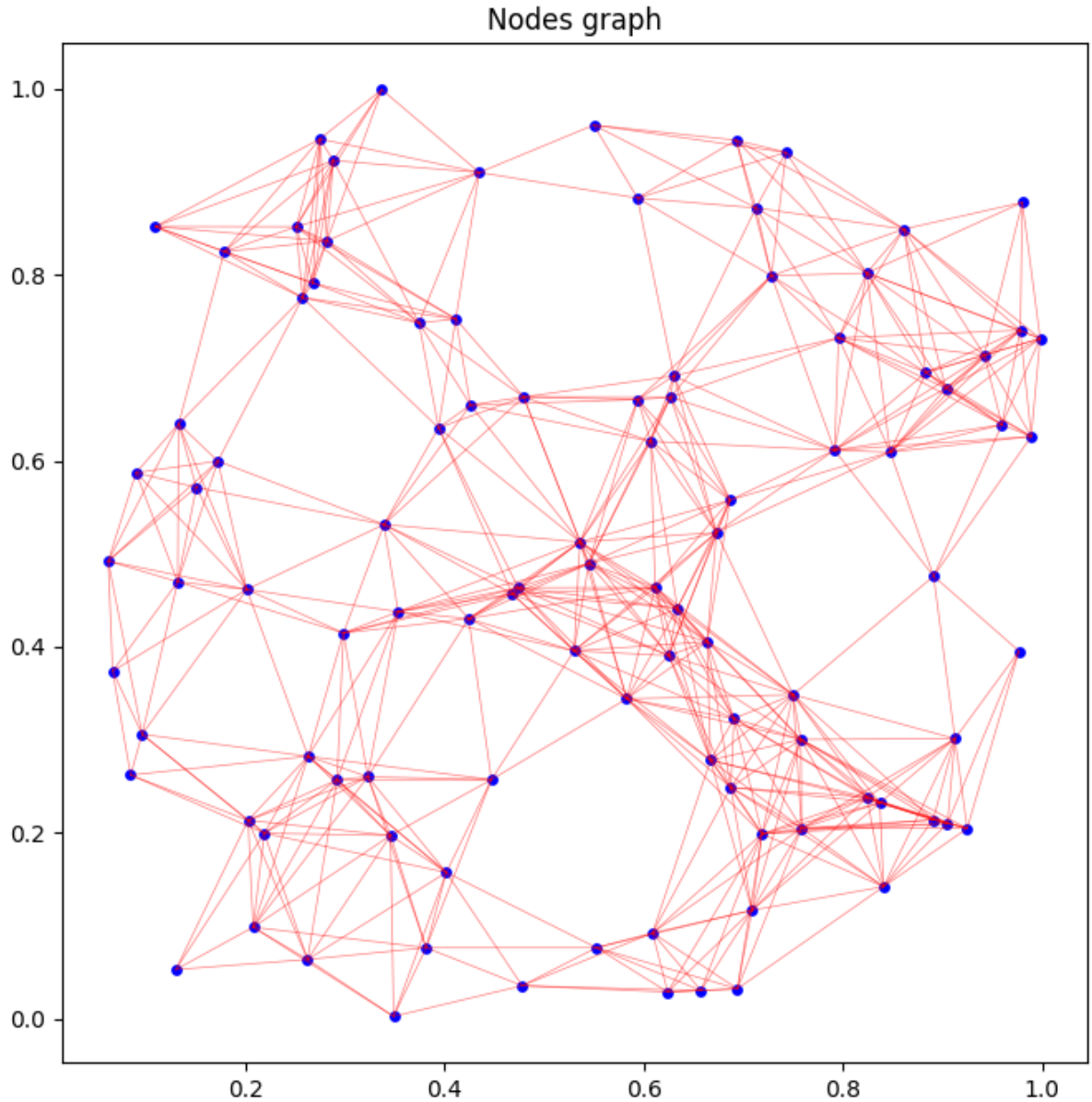

Figure 2: Graph of the connected network, $N = 100$, $r = 0.2$

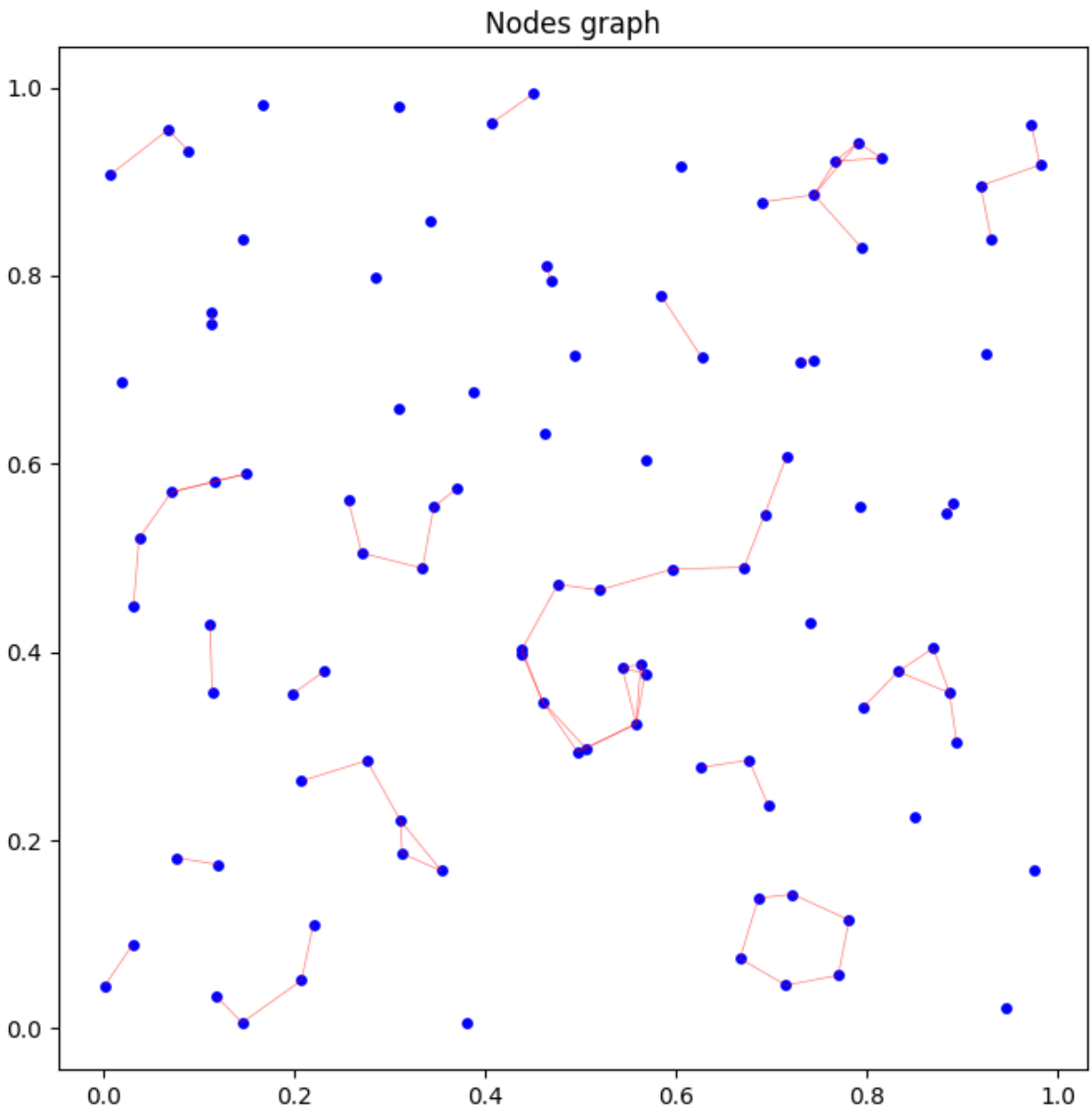

Figure 3: Graph of the not connected network, $N = 100$, $r = 0.08$

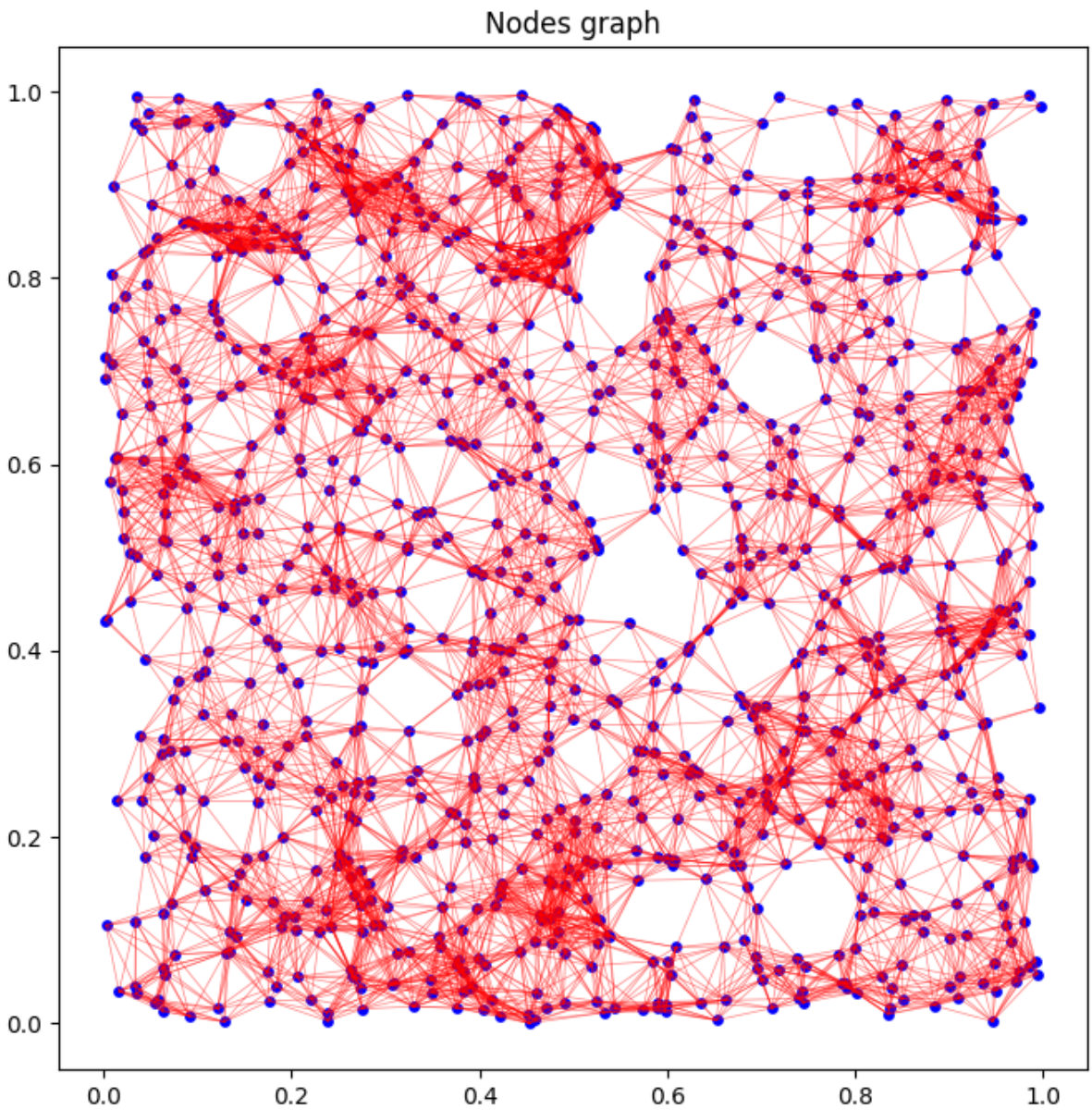

Figure 4: Graph of the connected network, $N = 1000$, $r = 0.08$

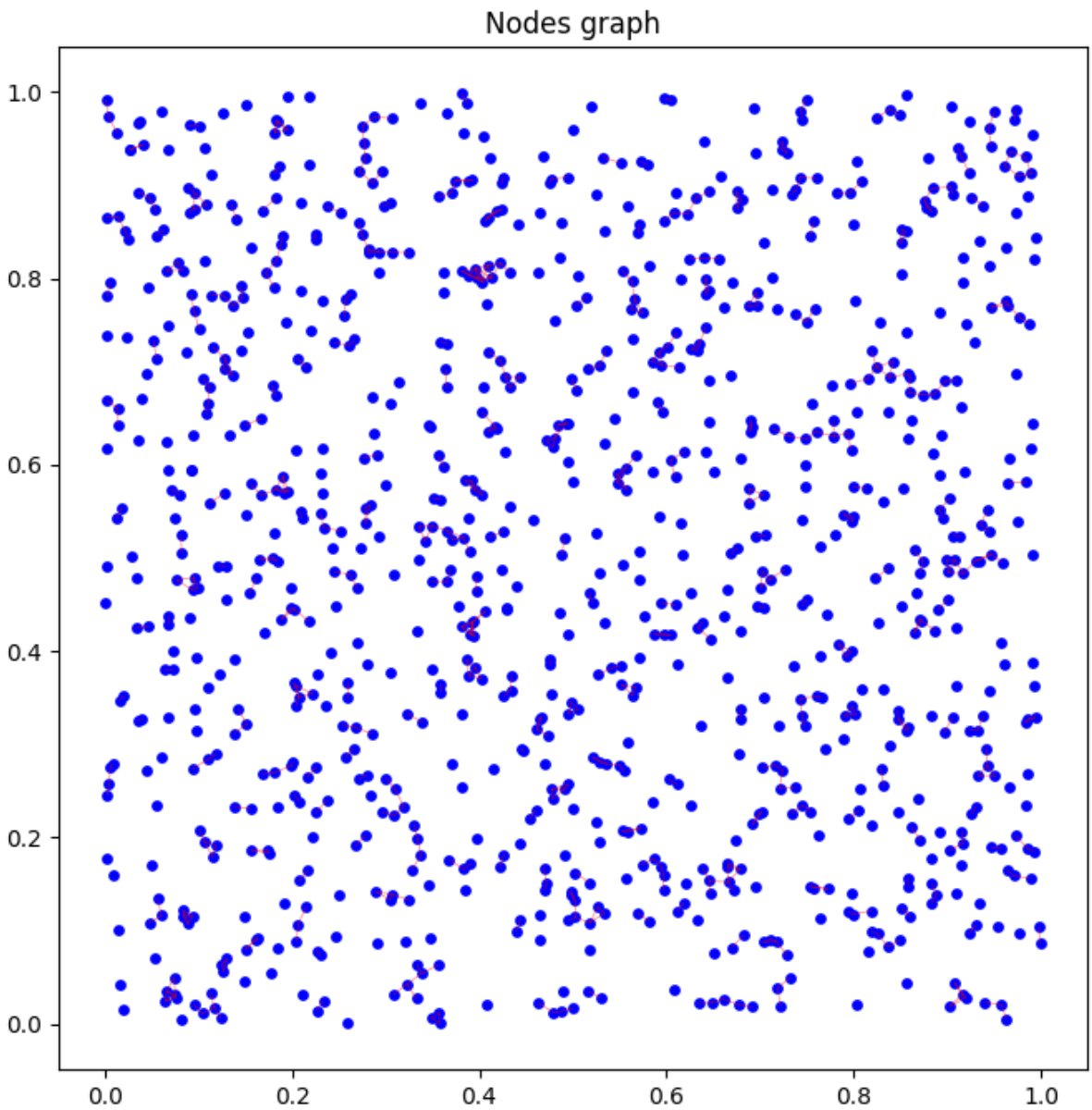

Figure 5: Graph of the not connected network, $N = 1000$, $r = 0.02$

2 Watts-Strogazt model

2.1 Definition

The Watts-Strogatz model is a random graph generation algorithm designed to produce networks with "small-world" properties. Introduced by Duncan J. Watts and Steven Strogatz in 1998, the model was developed to resolve a major limitation in traditional graph theory: it successfully bridges the gap between highly structured regular lattices and completely random graphs. It was created to mathematically replicate two key structural properties frequently observed in real world complex systems, such as social networks, power grids or the human brain.

2.2 Graph construction

Second task in the list states the following:

> - Start with N nodes arranged in a ring, where each node is connected to its k nearest neighbors ($k/2$ on each side; assume k is even).
> - For each edge (i, j), rewire it with probability p remove the edge (i, j) and connect node i to a randomly chosen node (excluding self-loops and duplicate edges).

In order to meet the requirements of this task, the following Python code has been implemented:

```
edges = []
for i in range(N):
    for d in range(1, k//2 + 1):
        r_neighbor = (i + d) % N
        edges.append((i, r_neighbor))
r_edges = []
for i, j in edges:
    if random.random() < p:
        new_j = random.randint(0, N - 1)
        while new_j == i or (i, new_j) in r_edges or (new_j, i) in
           r_edges:
            new_j = random.randint(0, N - 1)
        r_edges.append((i, new_j))
    else:
        r_edges.append((i, j))
graph = {i: [] for i in range(N)}
for i, j in r_edges:
    graph[i].append(j)
    graph[j].append(i)
# graph construction
def node_distances(init_node, graph):
    distances = {node: -1 for node in graph} # -1 means not visited
    distances[init_node] = 0 # first node
```

```
    queue = deque([init_node])
    while queue:
        curr_node = queue.popleft()
        for neighbor in graph[curr_node]:
            if distances[neighbor] == -1:
                distances[neighbor] = distances[curr_node] + 1
                queue.append(neighbor)
    return distances
```

2.3 Analysis

The purpose of analysis is to compute the average length of the shortest path for $N = 1000$, $k = 8$ and $\beta = [0.0, 0.01, 0.2]$, which can be expressed as a mathematical formula:

$$\langle l \rangle = \frac{1}{|V|(|V|-1)} \sum_{i \neq j} d(i, j).$$

```
Results for beta = 0.0 :
Average Shortest Path: 62.93793793793794
Results for beta = 0.01 :
Average Shortest Path: 10.703817817817818
Results for beta = 0.2 :
Average Shortest Path: 4.3609009009009005
```

As the results show, at $\beta = 0.0$ the network exists as a highly ordered ring lattice with strictly local connections, resulting in a predictably large average path length of roughly 62.94. However, introducing merely a $\beta = 0.01$ edge rewiring probability causes this path length to plummet dramatically to 10.70. This precipitous, non-linear drop is the defining characteristic of the Watts-Strogatz model, as it proves that only a microscopic fraction of long range shortcut edges is required to drastically shrink the global distance between any two nodes in a system. As randomness increases further to $\beta = 0.2$, the average path length continues to decrease to 4.36, demonstrating the network's steady transition toward the highly efficient short path topology characteristic of fully random graphs.

3 Graph analysis with NetworkX

3.1 Description

For the structural analysis of a real world network in Task 3, the "Bitcoin OTC" dataset was selected from the Stanford SNAP repository. This dataset captures a trust web of individuals trading on the Bitcoin OTC platform, a system inherently designed to establish reputation and prevent fraud among anonymous users. By its nature, the data forms an explicit, weighted, signed, and directed network, where edges represent transactions and are weighted by a trust rating scaling from -10 (total distrust) to +10 (total trust).

3.2 Analysis

By computing the degree of each node using $G.degree$ a following histogram of the degree distribution could be created:

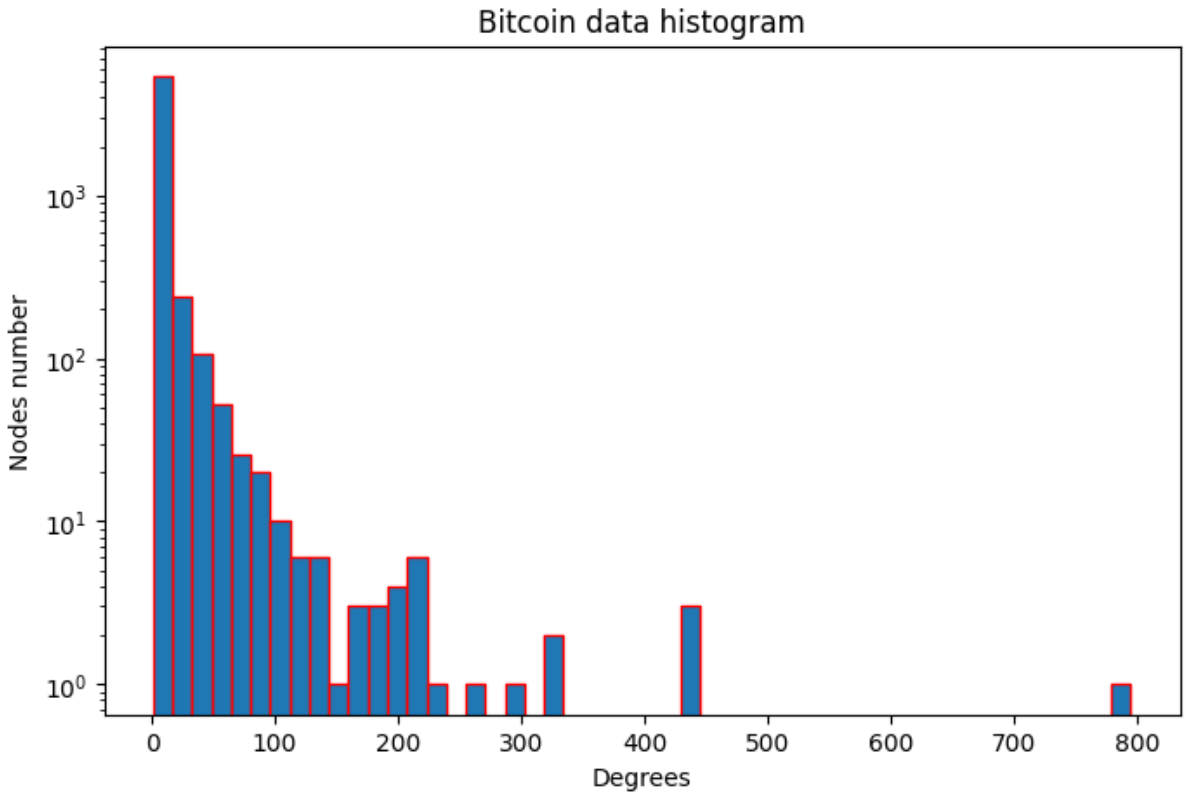

Figure 6: Degree distribution

Further analysis (comprised of computing the average shortest path length using $nx.average_shortest_path_length$ and average clustering coefficient using $nx.average_clustering$) has resulted in the following results:

```
Number of nodes in largest component: 5875
Average shortest path length: 3.5710878230065415
Average clustering coefficient: 0.17750449405289492
```

3.3 Visualization

For this visualization showing top 100 nodes with highest degrees, a Kamada-Kawai layout was chosen.

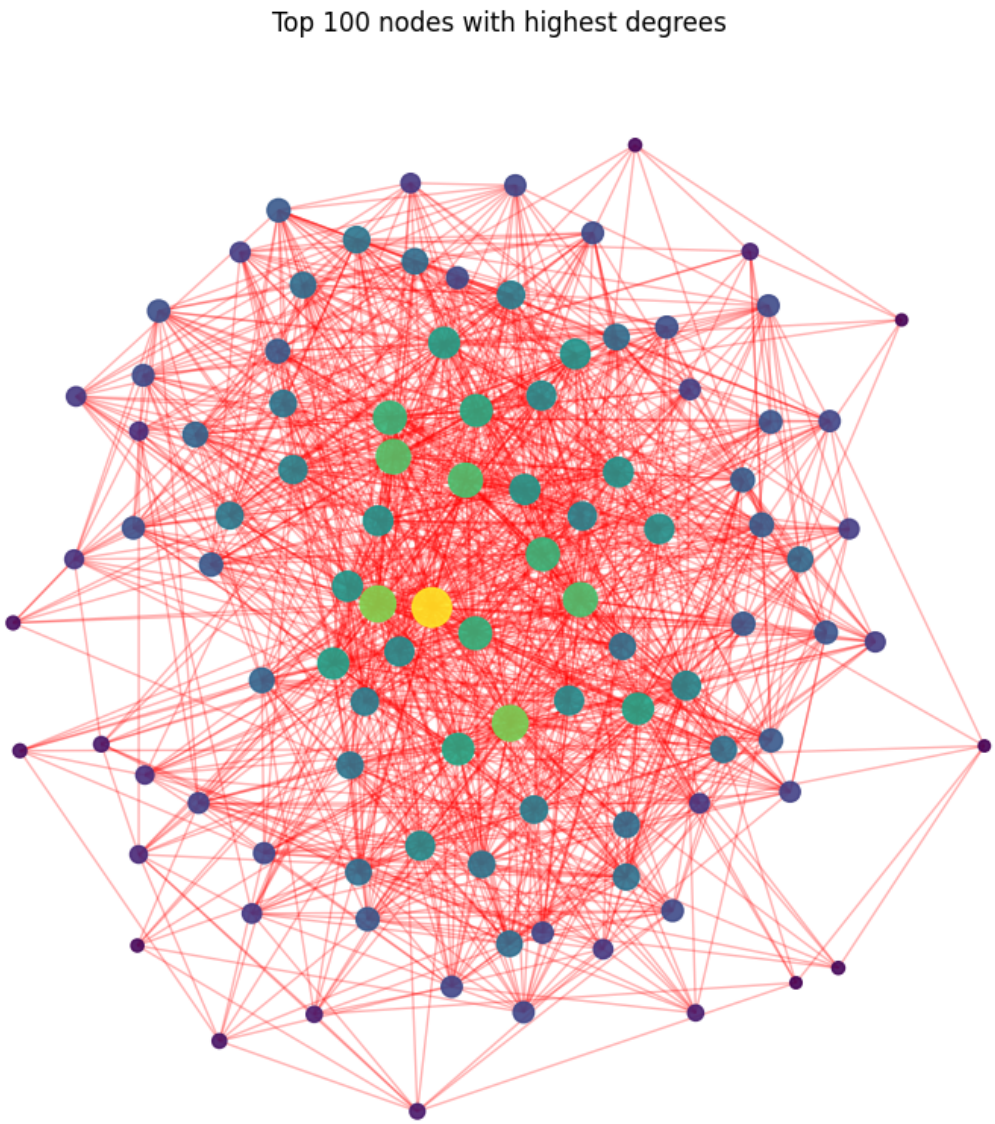

Figure 7: Network visualization